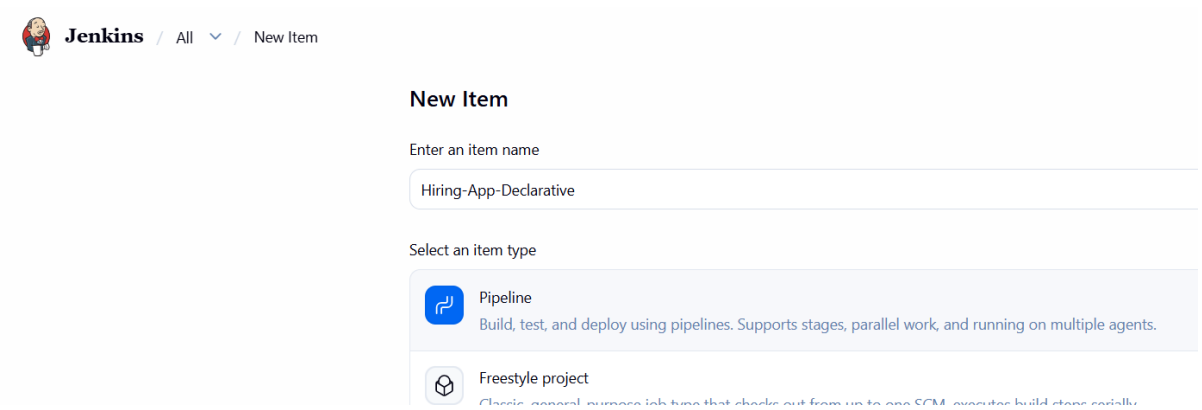


Jenkins challenge

Use the below code and implement our CICD stages.

- **Source Code:** <https://github.com/betawins/hiring-app.git>
 - Repository contains the application code for the hiring app.
 - **Create one Declarative pipeline job**
- Created a new item in Jenkins



Jenkins / All / New Item

New Item

Enter an item name

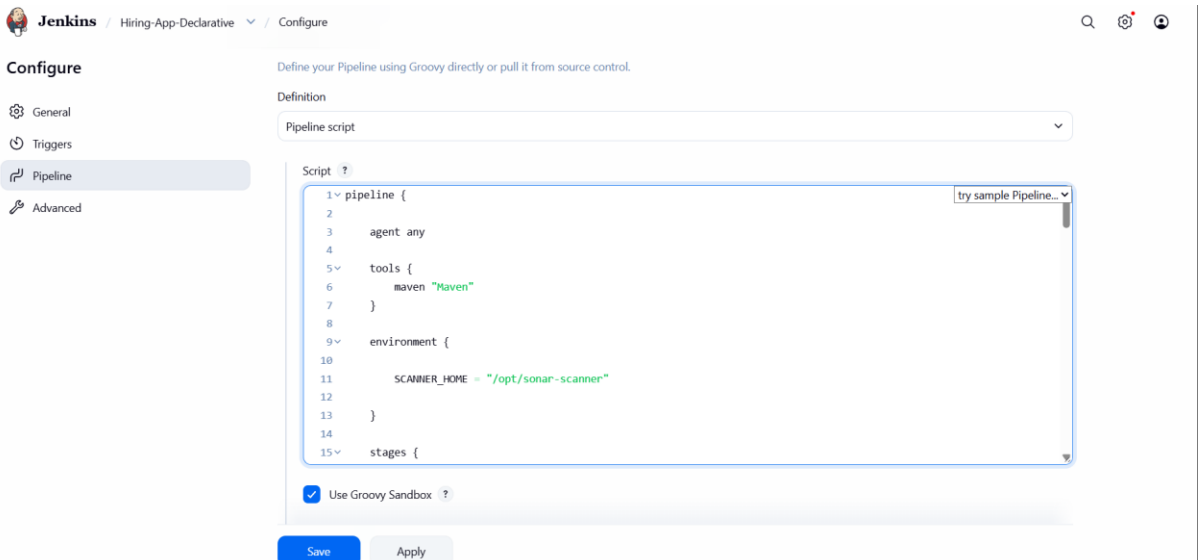
Hiring-App-Declarative

Select an item type

 Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 Freestyle project
Classic general-purpose job type that checks out from up to one SCM, executes build steps serially, and can trigger other jobs.

- Selected pipeline and written pipeline script and save



Jenkins / Hiring-App-Declarative / Configure

Configure

General
Triggers
Pipeline
Advanced

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script

```
1 pipeline {  
2  
3   agent any  
4  
5   tools {  
6     maven "Maven"  
7   }  
8  
9   environment {  
10  
11     SCANNER_HOME = "/opt/sonar-scanner"  
12  
13   }  
14  
15   stages {
```

☒ Use Groovy Sandbox

Save Apply

Script

```
pipeline {  
  
    agent any  
  
    tools {  
        maven "Maven"  
    }  
  
    environment {  
  
        SCANNER_HOME = "/opt/sonar-scanner"  
  
    }  
  
    stages {  
  
        stage('Git Clone'){  
  
            steps {  
  
                git branch: 'main',  
                url: 'https://github.com/betawins/hiring-app.git'  
  
            }  
  
        }  
  
        stage('SonarQube Analysis') {  
  
            steps {  
  
                sh '''  
                $SCANNER_HOME/bin/sonar-scanner \  
                -Dsonar.projectKey=hiring-app \  
                -Dsonar.projectName=hiring-app \  
                -Dsonar.sources=. \  
                -Dsonar.java.binaries=.  
                '''  
  
            }  
  
        }  
  
        stage('Maven Build') {  
  
            steps {  
  
                sh 'mvn clean package'
```

```

    }
}

stage('Nexus Upload') {

    steps {

        script {

            def warFile = findFiles(glob: 'target/*.war')[0].path

            nexusArtifactUploader(
                nexusVersion: 'nexus3',
                protocol: 'http',
                nexusUrl: '3.90.36.139:8081',
                groupId: 'com.hiring',
                version: "${BUILD_NUMBER}",
                repository: 'hiring-releases',
                credentialsId: 'nexus',
                artifacts: [
                    [
                        artifactId: 'hiring-app',
                        classifier: "",
                        file: warFile,
                        type: 'war'
                    ]
                ]
            )

        }

    }

}

stage('Deploy On Tomcat') {

    steps {

        script {

            def warFile = findFiles(glob: 'target/*.war')[0].path

            deploy adapters: [
                tomcat9(
                    credentialsId: 'tomcat',
                    path: "",
                    url: 'http://54.242.189.233:8080/manager/text'
                )
            ],

```

```

        contextPath: 'hiring',
        war: warFile
    }
}

}

}

post {

    success {

        slackSend(
            channel: 'C0B5EQ639SQ',
            color: 'good',
            message: "SUCCESS: ${env.JOB_NAME} Build ${env.BUILD_NUMBER}"
        )

    }

    failure {
        slackSend(
            channel: 'C0B5EQ639SQ',
            color: 'danger',
            message: "FAILED: ${env.JOB_NAME} Build ${env.BUILD_NUMBER}"
        )

    }

}

}

```

➤ Success script



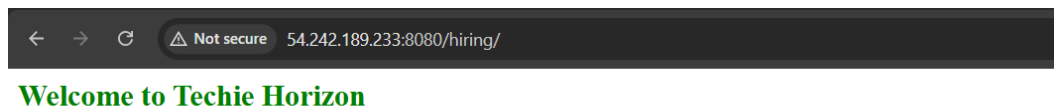
The image shows a Jenkins console output for a job named 'Hiring-App-Declarative'. The output indicates a successful deployment of the file 'hiring.war' to the path '/var/jenkins_home/workspace/Hiring-App-Declarative/target/hiring.war'. The console log includes standard Jenkins pipeline markers like '[Pipeline]', '[Pipeline] // script', '[Pipeline] // withEnv', '[Pipeline] stage', and '[Pipeline] End of Pipeline'. A specific step 'Slack Send Pipeline' is shown with its configuration: baseUrl is empty, teamDomain is 'arun-mm8244', channel is '#jenkins', color is 'good', botUser is false, tokenCredentialId is 'Slack', notifyCommitters is false, iconEmoji is empty, username is empty, and timestamp is empty. An error message follows: 'ERROR: Slack notification failed. See Jenkins logs for details.' The final status of the pipeline is 'Finished: SUCCESS'.

```

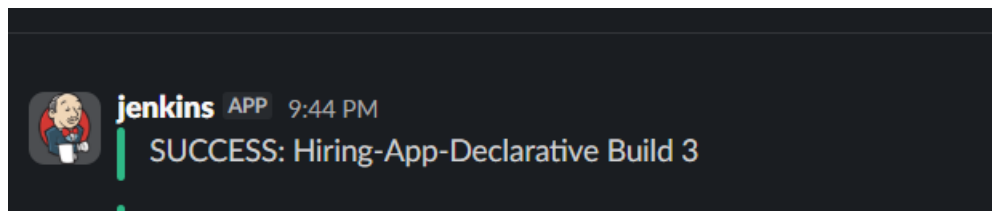
Deploying [/var/jenkins_home/workspace/Hiring-App-Declarative/target/hiring.war]
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: arun-mm8244, channel: #jenkins, color: good, botUser: false,
tokenCredentialId: Slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
ERROR: Slack notification failed. See Jenkins logs for details.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

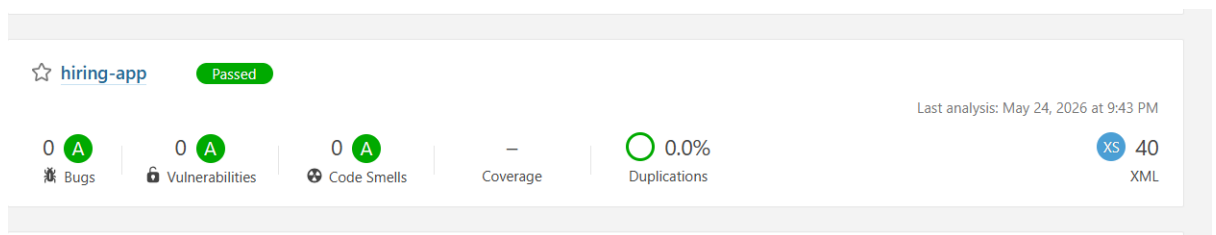
- Check in browser it is running



- Check in slack notification

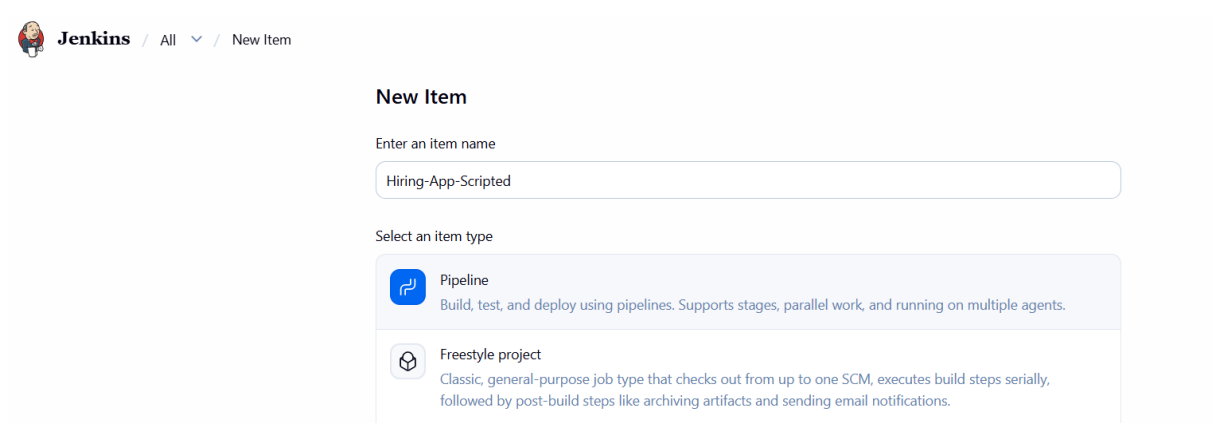


- Check in SonarQube

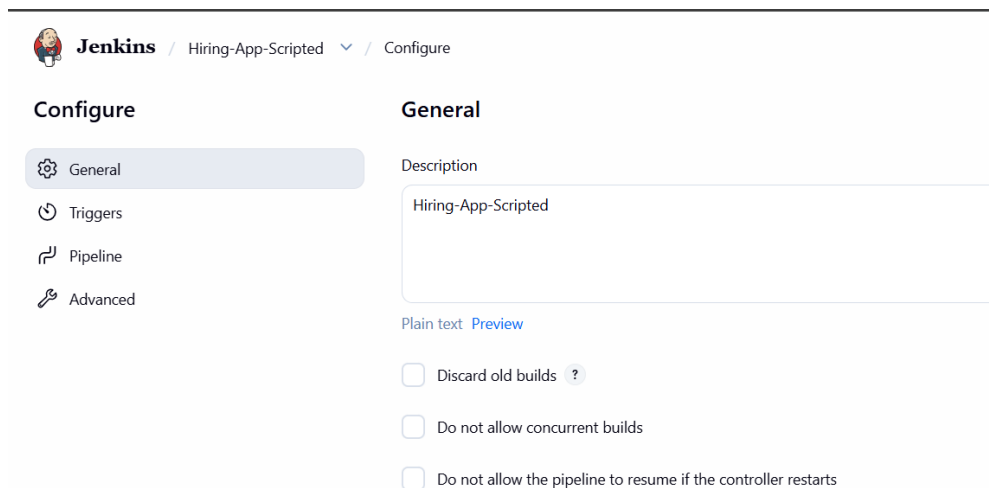


- **Create one Scripted pipeline job**

- Created a new item in Jenkins



➤ Selected Pipeline and written the scripted pipeline



The screenshot shows the Jenkins configuration interface. At the top, the breadcrumb navigation reads 'Jenkins / Hiring-App-Scripted / Configure'. On the left, under the 'Configure' heading, there is a sidebar with four options: 'General' (selected), 'Triggers', 'Pipeline', and 'Advanced'. The 'General' tab is active, showing a 'Description' field with the text 'Hiring-App-Scripted'. Below the description field, there are three unchecked checkboxes: 'Discard old builds', 'Do not allow concurrent builds', and 'Do not allow the pipeline to resume if the controller restarts'. There are also links for 'Plain text' and 'Preview'.

Script

```
node {  
  
    stage('Git Clone') {  
  
        git branch: 'main',  
  
        url: 'https://github.com/betawins/hiring-app.git'  
  
    }  
  
    stage('SonarQube Analysis') {  
  
        sh '''  
  
        /opt/sonar-scanner/bin/sonar-scanner \  
  
        -Dsonar.projectKey=hiring-scripted \  
  
        -Dsonar.projectName=hiring-scripted \  
  
        -Dsonar.sources=. \  
  
        -Dsonar.java.binaries=.  
  
        '''  
  
    }  
  
    stage('Maven Build') {
```

```
    sh 'mvn clean package'
}

stage('Nexus Upload') {
    script {
        sh 'ls target/*.war | head -1 > war.txt'

        def warFile = readFile('war.txt').trim()

        echo "WAR FILE: ${warFile}"

        nexusArtifactUploader(
            nexusVersion: 'nexus3',
            protocol: 'http',
            nexusUrl: '3.90.36.139:8081',
            groupId: 'com.hiring',
            version: "${BUILD_NUMBER}",
            repository: 'hiring-releases',
            credentialsId: 'nexus',
            artifacts: [
                [
                    artifactId: 'hiring-app',
                    classifier: '',
                    file: warFile,
                    type: 'war'
                ]
            ]
        )
    }
}
```

```

}

stage('Deploy On Tomcat') {

    script {

        sh 'ls target/*.war | head -1 > war.txt'

        def warFile = readFile('war.txt').trim()

        echo "Deploying WAR: ${warFile}"

        deploy adapters: [

            tomcat9(

                credentialsId: 'tomcat',
                path: '',
                url: 'http://54.242.189.233:8080/manager/text'
            )

        ],

        contextPath: 'hiring',
        war: warFile
    }
}
}

```

- After build now and check the status as success

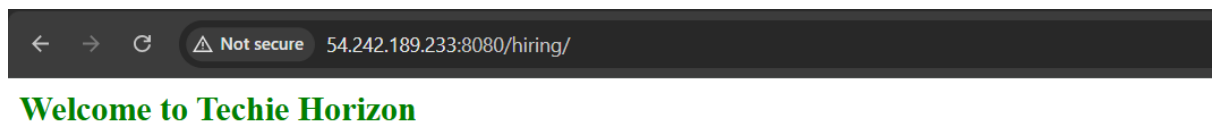

Jenkins / Hiring-App-Scripted ▾ / #2 ▾
🔍

```

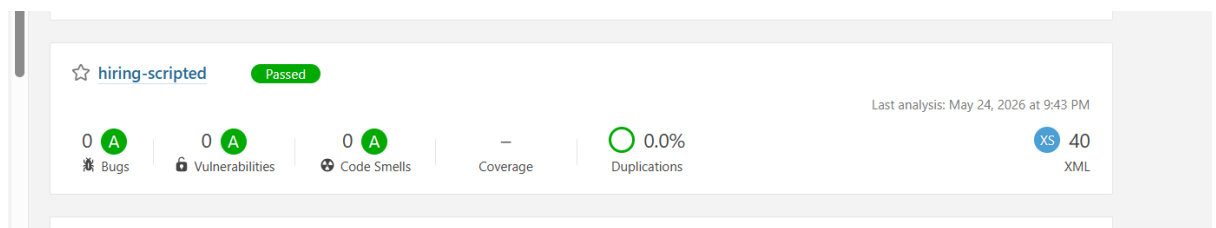
[Pipeline] deploy
[DeployPublisher][INFO] Attempting to deploy 1 war file(s)
[DeployPublisher][INFO] Deploying /var/jenkins_home/workspace/Hiring-App-Scripted/target/hiring.war to container Tomcat 9.x Remote with
context hiring
  Redeploying [/var/jenkins_home/workspace/Hiring-App-Scripted/target/hiring.war]
  Undeploying [/var/jenkins_home/workspace/Hiring-App-Scripted/target/hiring.war]
  Deploying [/var/jenkins_home/workspace/Hiring-App-Scripted/target/hiring.war]
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Slack Notification)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: arun-mmm8244, channel: #jenkins, color: good, botUser: fal
tokenCredentialId: Slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
ERROR: Slack notification failed. See Jenkins logs for details.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

- Check browser URL is running or not

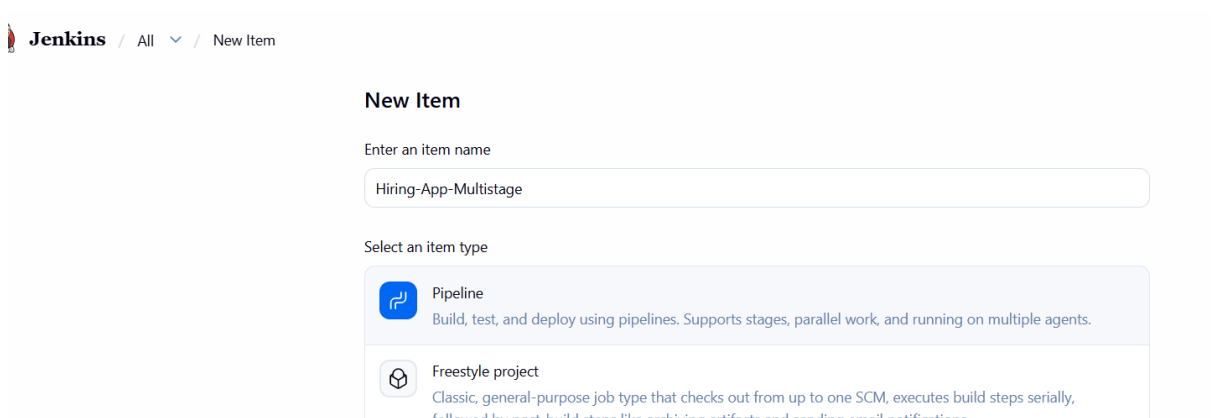


- Check in SonarQube

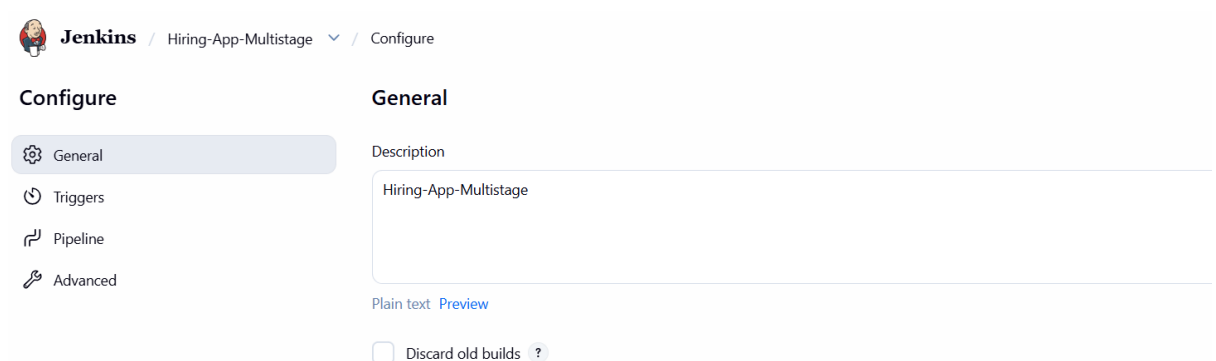


- **Create one multi stage pipeline job**

- Create a new item in Jenkins



- Selected pipeline and written script



- Script written and save it

```
pipeline {  
    agent any  
  
    tools {  
        maven "Maven"  
    }  
  
    environment {  
        SCANNER_HOME = "/opt/sonar-scanner"  
    }  
  
    stages {  
        stage('Git Clone') {  
            steps {  
                git branch: 'main',  
                url: 'https://github.com/betawins/hiring-app.git'  
            }  
        }  
  
        stage('SonarQube Analysis') {  
            steps {  
                sh '''  
                $SCANNER_HOME/bin/sonar-scanner \  
                -Dsonar.projectKey=hiring-multistage \  
                -Dsonar.projectName=hiring-multistage \  
                -Dsonar.sources=. \  
                -Dsonar.java.binaries=.  
                '''  
            }  
        }  
  
        stage('Compile') {  
            steps {
```

```
        sh 'mvn compile'
    }
}

stage('Test') {
    steps {
        sh 'mvn test'
    }
}

stage('Package') {
    steps {
        sh 'mvn package'
    }
}

stage ('Nexus Upload') {
    steps {
        script {
            def warFile = findFiles (glob: 'target/*.war')[0].path

            echo "WAR FILE: ${warFile}"

            nexusArtifactUploader(
                nexusVersion: 'nexus3',
                protocol: 'http',
                nexusUrl: '3.90.36.139:8081',
                groupId: 'com.hiring',
                version: "${BUILD_NUMBER}",
                repository: 'hiring-releases',
                credentialsId: 'nexus',
                artifacts: [
                    [
                        artifactId: 'hiring-app',
                        classifier: '',
                        file: warFile,
```

```

        type: 'war'
    ]
}
}

stage('Deploy On Tomcat') {

    steps {

        script {

            def warFile = findFiles(glob: 'target/*.war')[0].path

            deploy adapters: [
                tomcat9(
                    credentialsId: 'tomcat',
                    path: '',
                    url: 'http://54.242.189.233:8080/manager/text'
                )
            ],
            contextPath: 'hiring',
            war: warFile

        }

    }

}

post {

    success {

        slackSend(
            channel: 'C0B5EQ639SQ',
            color: 'good',
            message: "SUCCESS: ${env.JOB_NAME} Build
${env.BUILD_NUMBER}"
        )

    }

}

```

```

failure {

    slackSend(
        channel: 'C0B5EQ639SQ',
        color: 'danger',
        message: "FAILED: ${env.JOB_NAME} Build
${env.BUILD_NUMBER}"
    )

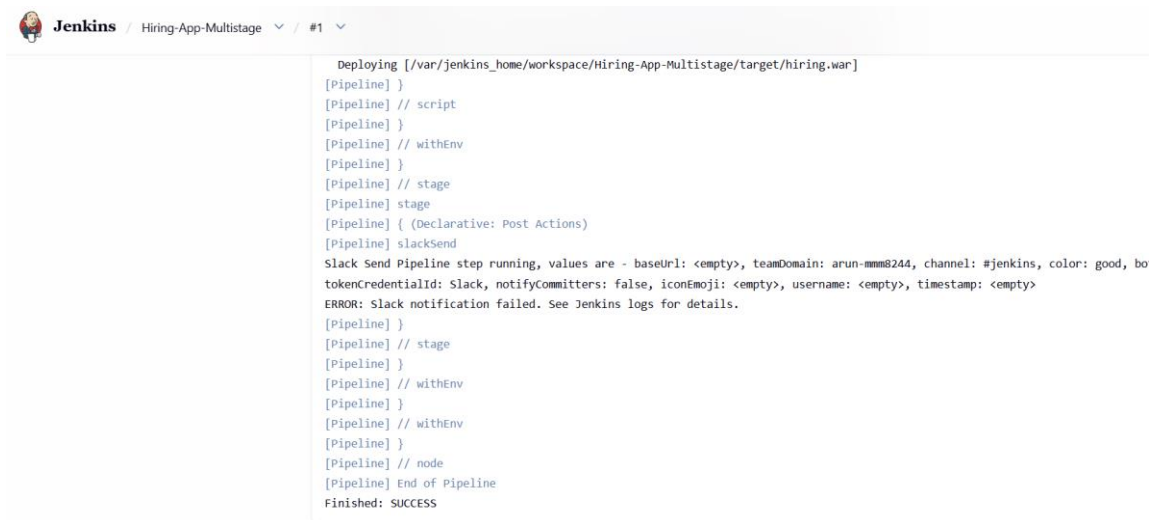
}

}

}

```

- Check console it is success



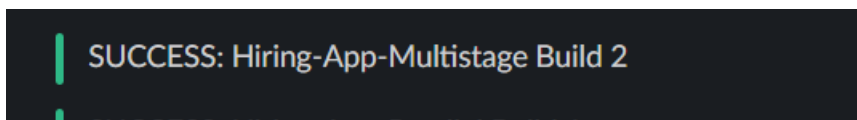
The image shows a Jenkins console output for a build named 'Hiring-App-Multistage'. The output indicates a successful deployment of 'hiring.war' to the workspace. However, there is an error in the 'slackSend' step: 'ERROR: Slack notification failed. See Jenkins logs for details.' The rest of the pipeline steps, including 'stage', 'withEnv', and 'node', complete successfully, resulting in a 'Finished: SUCCESS' status.

```

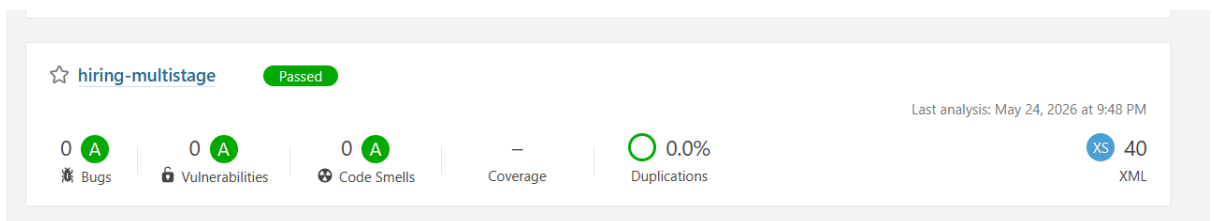
Deploying [/var/jenkins_home/workspace/Hiring-App-Multistage/target/hiring.war]
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: arun-mmm8244, channel: #jenkins, color: good, bo
tokenCredentialId: Slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
ERROR: Slack notification failed. See Jenkins logs for details.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

- Check in slack notification

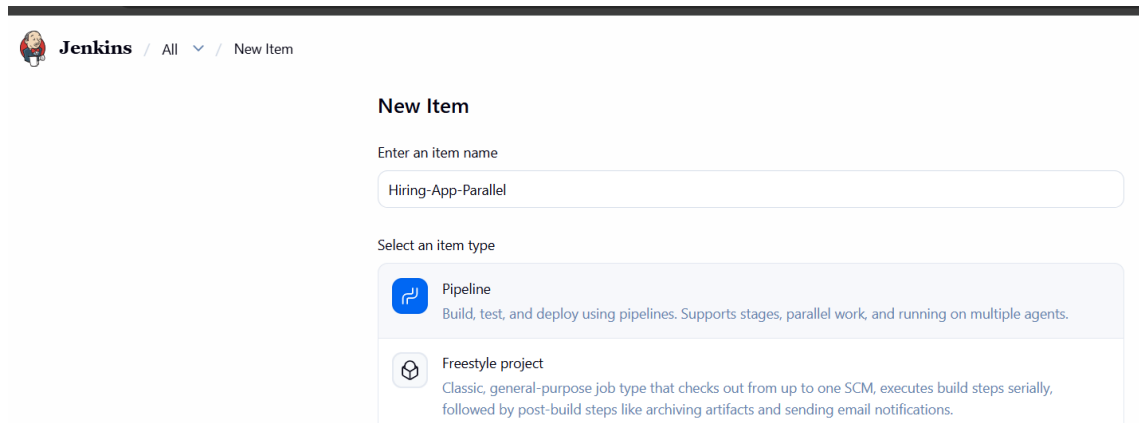


- Check in SonarQube

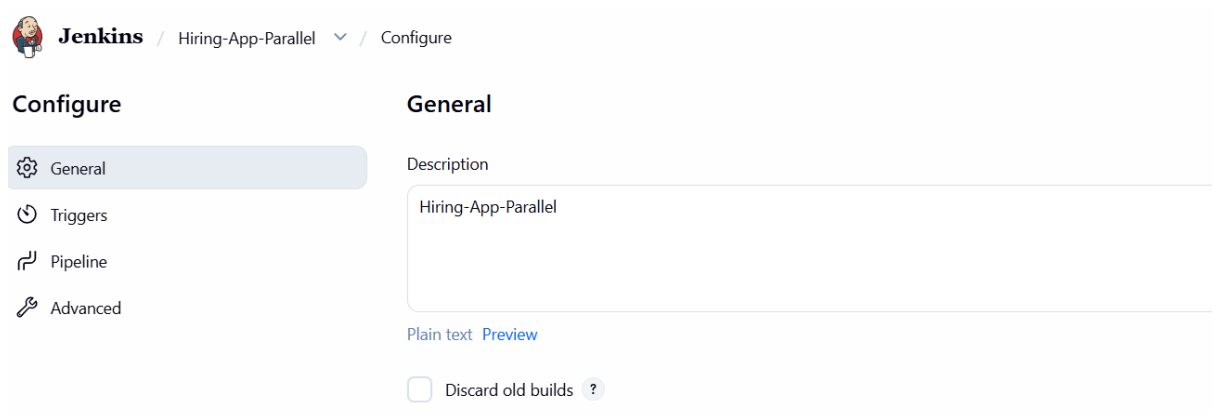


- **Create one parallel stage pipeline job**

➤ Create a new item in Jenkins



➤ Selected pipeline and written script



➤ Script

```
pipeline {  
  
  agent any  
  
  tools {  
  
    maven "Maven"  
  
  }  
  
  environment {  
  
    SCANNER_HOME = "/opt/sonar-scanner"  
  
  }  
}
```

```
stages {  
    stage('Git Clone') {  
        steps {  
            git branch: 'main',  
            url: 'https://github.com/betawins/hiring-app.git'  
        }  
    }  
  
    stage('Parallel Execution') {  
        parallel {  
            stage('SonarQube Analysis') {  
                steps {  
                    sh '''  
                        $SCANNER_HOME/bin/sonar-scanner \  
                        -Dsonar.projectKey=hiring-parallel \  
                        -Dsonar.projectName=hiring-parallel \  
                        -Dsonar.sources=. \  
                        -Dsonar.java.binaries=.  
                    '''  
                }  
            }  
  
            stage('Maven Build') {  
                steps {  
                    sh 'mvn clean package'  
                }  
            }  
        }  
    }  
}
```

```
}  
  
}  
  
stage('Nexus Upload') {  
    steps {  
        script {  
            def warFile = findFiles(glob: 'target/*.war')[0].path  
  
            echo "WAR FILE: ${warFile}"  
  
            nexusArtifactUploader(  
                nexusVersion: 'nexus3',  
                protocol: 'http',  
                nexusUrl: '3.90.36.139:8081',  
                groupId: 'com.hiring',  
                version: "${BUILD_NUMBER}",  
                repository: 'hiring-releases',  
                credentialsId: 'nexus',  
                artifacts: [  
                    [  
                        artifactId: 'hiring-app',  
                        classifier: '',  
                        file: warFile,  
                        type: 'war'  
                    ]  
                ]  
            )  
        }  
    }  
}
```

```

}

}

stage('Deploy On Tomcat') {

    steps {

        script {

            def warFile = findFiles(glob: 'target/*.war')[0].path

            deploy adapters: [

                tomcat9(

                    credentialsId: 'tomcat',

                    path: '',

                    url: 'http://54.242.189.233:8080/manager/text'

                )

            ],

            contextPath: 'hiring',

            war: warFile

        }

    }

}

post {

    success {

        slackSend(

            channel: 'C0B5EQ639SQ',

            color: 'good',

            message: "SUCCESS: ${env.JOB_NAME} Build  
${env.BUILD_NUMBER}"

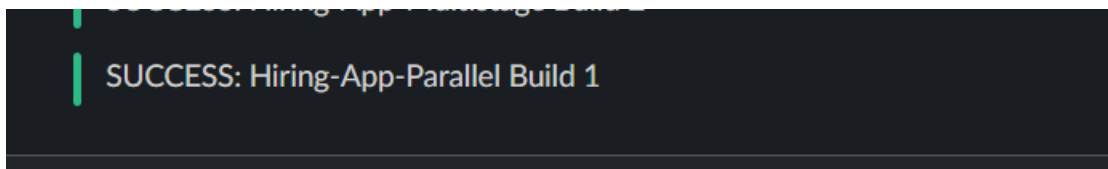
```

```

    )
  }
  failure {
    slackSend(
      channel: 'C0B5EQ639SQ',
      color: 'danger',
      message: "FAILED: ${env.JOB_NAME} Build
${env.BUILD_NUMBER}"
    )
  }
}
}
}

```

- After success of script check in slack for notification



- Check in SonarQube also

